🚀 Lecture 04.05: List Mutator Methods & Algorithmic Complexity

📌 Executive Architectural Summary

CPython provides optimized C-level methods for list manipulation. Understanding their algorithmic time complexities ($\mathcal{O}(1)$ vs $\mathcal{O}(N)$ vs $\mathcal{O}(N \log N)$) is essential for high-performance numerical software.

🔬 Core Engineering Mechanics

1. Methods Taxonomy & Complexity

Method	Operation	Complexity	Description
`append(x)`	Append element to tail	$\mathcal{O}(1)$ amortized	Appends to pre-allocated buffer
`extend(iterable)`	Append iterable elements	$\mathcal{O}(K)$	Resizes buffer and copies pointers
`insert(i, x)`	Insert at index i	$\mathcal{O}(N)$	Shifts elements from i to $N-1$ rightward
`remove(x)`	Remove first occurrence of x	$\mathcal{O}(N)$	Searches linearly, then shifts left
`pop(i)`	Remove and return element at i	$\mathcal{O}(1)$ at tail, $\mathcal{O}(N)$ elsewhere	Tail pop involves no shifting
`sort()`	Sort list in-place	$\mathcal{O}(N \log N)$	Timsort adaptive mergesort
`reverse()`	Reverse list in-place	$\mathcal{O}(N)$	Two-pointer swap traversal

---

💻 Production Python Implementation

```
nums = [40, 10, 30, 20, 50]

# Append vs Extend
nums.append(60)
nums.extend([70, 80])
print("After append & extend:", nums)

# Insert & Pop
nums.insert(0, 5) # O(N) shift
popped_val = nums.pop() # O(1) tail removal
print(f"Popped: {popped_val} | Current: {nums}")

# In-Place Sort (Timsort)
nums.sort()
print("Sorted In-Place:", nums)
nums.reverse()
print("Reversed In-Place:", nums)
```